

# The weekend decision

| Outlook      | Temp        | Humidity    | Wind          | Play?      |
|--------------|-------------|-------------|---------------|------------|
| Sunny        | Hot         | High        | Weak          | No         |
| Sunny        | Hot         | High        | Strong        | No         |
| Overcast     | Hot         | High        | Weak          | Yes        |
| Rain         | Mild        | High        | Weak          | Yes        |
| Rain         | Cool        | Normal      | Weak          | Yes        |
| Rain         | Cool        | Normal      | Strong        | No         |
| Overcast     | Cool        | Normal      | Strong        | Yes        |
| Sunny        | Mild        | High        | Weak          | No         |
| Sunny        | Cool        | Normal      | Weak          | Yes        |
| Rain         | Mild        | Normal      | Weak          | Yes        |
| Sunny        | Mild        | Normal      | Strong        | Yes        |
| Overcast     | Mild        | High        | Strong        | Yes        |
| Overcast     | Hot         | Normal      | Weak          | Yes        |
| Rain         | Mild        | High        | Strong        | No         |
| <b>Sunny</b> | <b>Cool</b> | <b>High</b> | <b>Strong</b> | <b>???</b> |

14 past weekends: did we **go out and play** (Yes/No), given the weather?

Today is **Sunny, Cool, High humidity, Strong wind**.

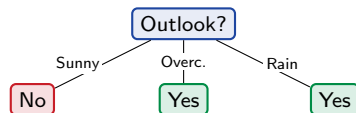
*Would you go out?*

This is the classic *Play Tennis* dataset (9 Yes, 5 No) – our Armenian “*go to Sevan this weekend?*” We will learn to decide by asking a few yes/no questions.

## Predict from one feature

Simplest possible model: look at **one** feature and vote the majority. Take **Outlook**:

| Outlook  | Yes | No | majority    |
|----------|-----|----|-------------|
| Sunny    | 2   | 3  | No          |
| Overcast | 4   | 0  | Yes (pure!) |
| Rain     | 3   | 2  | Yes         |



A tree with a **single** decision node is a **decision stump**. Overcast is already *pure* (all Yes) – no need to ask anything else there. (We will reuse “stump” for AdaBoost in [19].)

# Outline

What a tree is

Why we need impurity

Impurity measures

Growing a tree (CART)

Overfitting and pruning

In practice and bridge

## What a tree is

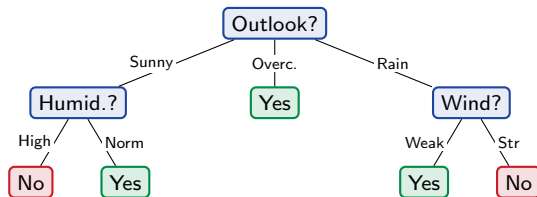
A model you can read: a chain of yes/no questions down to an answer.

## Sunny and Rain are still mixed – ask another question

Sunny (2+, 3-) and Rain (3+, 2-) are impure. Split each again – Sunny by **Humidity**, Rain by **Wind**:

A tree is just **nested if/else** rules:

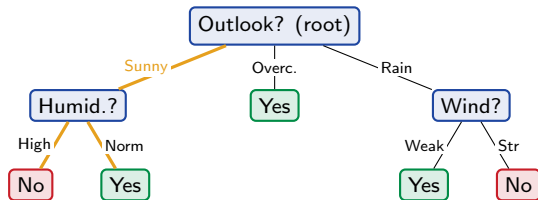
```
if Outlook = Overcast:  Yes
elif Outlook = Sunny:
    if Humidity = High:  No else:  Yes
elif Outlook = Rain:
    if Wind = Strong:    No else:  Yes
```



The if/else block and the tree are *the same model* – two ways to write it.

# Anatomy of a tree

- ▶ **Root:** the first question (top).
- ▶ **Internal node:** a question on one feature.
- ▶ **Branch:** an answer to that question.
- ▶ **Leaf:** a final prediction (no more questions).
- ▶ **Depth:** longest root-to-leaf path.



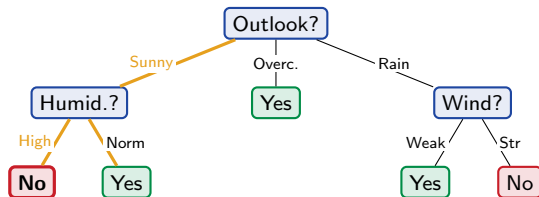
**Prediction** = walk from the root,  
follow the branch that matches the row,  
output the leaf you land in.

# Predicting today (Sunny, Cool, High, Strong)

Walk the tree:

1. Outlook = **Sunny** → go left.
2. Humidity = **High** → leaf.
3. Predict **No**.

That “No” is backed by **3 training rows** (all Sunny + High weekends said No). We did not even need Temperature or Wind.



## Why we need impurity

To grow a tree we must score a split: how much purer did the node get?

## Feature order matters

We started with Outlook. What if we had started with **Temperature**?

### Outlook first (our tree)

- ▶ Today → Sunny → High → **No**.
- ▶ Leaf backed by **3 rows**.

### Temperature first (another tree)

- ▶ Today → Cool → Sunny → ... only **1 matching row** (D9, which was Normal/Weak) → **Yes**.

Same data, **different feature order** ⇒ **different tree** ⇒ **different prediction** for today (No vs Yes). Which one should we trust?

## Predict-first: which tree is better?

One tree predicts **No** from a leaf resting on **3** weekends; the other predicts **Yes** from a leaf resting on **1** weekend (that also differed in humidity and wind).

**Which prediction would you trust?**

## Predict-first: which tree is better?

One tree predicts **No** from a leaf resting on **3** weekends; the other predicts **Yes** from a leaf resting on **1** weekend (that also differed in humidity and wind).

### Which prediction would you trust?

The one backed by **more rows**. A leaf covering many consistent examples generalizes; a leaf covering one example is memorizing.

**Smaller trees usually generalize better** – their leaves rest on more rows (higher coverage). A good split order reaches *pure, well-covered* leaves *sooner*. (We will make this precise as overfitting in Section 5.)

So: split on whatever increases purity the most

We want pure leaves *sooner*  $\Rightarrow$  at each node, pick the feature whose split makes the children **as pure as possible**.

Recall: Outlook's Overcast branch was *instantly pure* (4 Yes, 0 No). That is why Outlook is a great first question.

To choose automatically, we need a number that measures how **impure** (mixed) a set of labels is. Two standard ones: **Gini** and **entropy**.

## Impurity measures

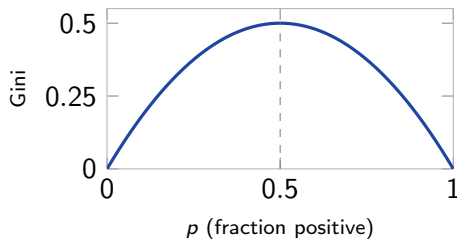
Gini and entropy: two ways to put a number on how mixed a node is.

## Gini impurity (and its famous cousin)

$$\text{Gini} = 1 - \sum_k p_k^2 \quad (\text{binary: } 2p(1-p)).$$

**Cleanest reading:** pull *two* points at random from the node – Gini is the probability they are **different** classes.

- ▶ pure (all one class):  $1 - 1 = 0$  (always the same).
- ▶ 80/20:  $1 - (0.8^2 + 0.2^2) = 0.32$ .
- ▶ 50/50:  $1 - (0.5^2 + 0.5^2) = 0.5$  (maximally mixed).



**The economic cousin.** Named after Corrado Gini (1912). The economics *Gini index* of income inequality is a sibling measure – both ask “*how different are two random members of a population?*” (here, their class; in economics, their income). Same idea, different formula.

# Entropy and information gain

**Entropy** (in bits) is another impurity measure:

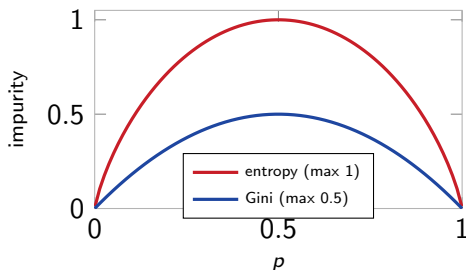
$$H = -p \log_2 p - (1 - p) \log_2 (1 - p).$$

The split's value is the **information gain** – impurity removed:

$$\text{IG} = H_{\text{parent}} - \sum_{\text{child } c} \frac{n_c}{n} H_c.$$

Same idea for Gini (“Gini gain”). Pick the split with the biggest drop.

Both peak at  $p = 0.5$  and vanish at the pure ends – they mostly agree.



## Worked by hand: Gini gain of the Outlook split

**Parent** node (9 Yes, 5 No):

$$\text{Gini} = 1 - \left(\frac{9}{14}\right)^2 - \left(\frac{5}{14}\right)^2 = 1 - 0.413 - 0.128 = \mathbf{0.459}.$$

**Children** after splitting on Outlook:

| child    | counts      | Gini                                                                 |
|----------|-------------|----------------------------------------------------------------------|
| Sunny    | 2 Yes, 3 No | $1 - \left(\frac{2}{5}\right)^2 - \left(\frac{3}{5}\right)^2 = 0.48$ |
| Overcast | 4 Yes, 0 No | $1 - 1 - 0 = 0$ (pure!)                                              |
| Rain     | 3 Yes, 2 No | $1 - \left(\frac{3}{5}\right)^2 - \left(\frac{2}{5}\right)^2 = 0.48$ |

**Weighted child impurity** (weight = child rows /14), then the gain:

$$\frac{5}{14}(0.48) + \frac{4}{14}(0) + \frac{5}{14}(0.48) = 0.343 \Rightarrow \text{gain} = 0.459 - 0.343 = \mathbf{0.116}.$$

Overcast is a *free pure leaf* (Gini = 0) – that is why Outlook scores so well. Do this for every feature; keep the biggest gain.

## Worked split: which feature is the best root?

Parent entropy (9 Yes, 5 No):  $H = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} = 0.940$  bits. Information gain of each feature:

| split on       | information gain                         |
|----------------|------------------------------------------|
| <b>Outlook</b> | <b>0.247</b> ← winner (Overcast is pure) |
| Humidity       | 0.152                                    |
| Wind           | 0.048                                    |
| Temperature    | 0.029                                    |

**Outlook** removes the most impurity, so CART makes it the root – exactly the tree we drew by hand.

These numbers use **entropy / information gain**. sklearn defaults to **Gini** – it picks the same winner here and gives a near-identical tree. Do not be surprised the criterion silently changes between this slide and the sklearn one.

## The trap: information gain loves many-valued features

**Predict-first:** add a Day column (14 distinct dates, one per row) to the weather data.  
Which feature does information gain pick as the root?

## The trap: information gain loves many-valued features

**Predict-first:** add a Day column (14 distinct dates, one per row) to the weather data. Which feature does information gain pick as the root?

**Day wins** - and it is useless. Splitting on Day gives 14 pure single-row leaves, so its children have zero impurity:

$$\text{IG}(\text{Day}) = H_{\text{parent}} - 0 = 0.940 \text{ bits} > \text{IG}(\text{Outlook}) = 0.247.$$

A feature with **many distinct values** gets more chances to carve the data into pure single-row leaves, so raw gain is **biased toward high-cardinality features** - even an ID that never generalizes. Fixes: **gain ratio** (C4.5) normalizes by the split's own entropy; CART's binary splits soften it; and it is why **high-cardinality categoricals** need care (CatBoost's target encoding, [20]).

## Regression trees: same idea, numeric target

Predict a *number* (apartment rent from area). Split to reduce **spread** (variance / SSE) instead of class impurity; each leaf predicts the **mean** of its rows.

**Predict-first:** what does the tree predict for an area *between* two training points?

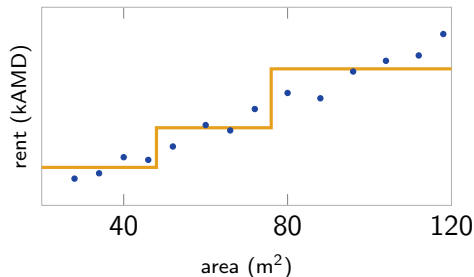
## Regression trees: same idea, numeric target

Predict a *number* (apartment rent from area). Split to reduce **spread** (variance / SSE) instead of class impurity; each leaf predicts the **mean** of its rows.

**Predict-first:** what does the tree predict for an area *between* two training points?

The leaf **mean** – a **flat step**, not a smooth line. A tree's prediction is piecewise-constant.

Deeper tree  $\Rightarrow$  more steps  $\Rightarrow$  closer fit (and, eventually, overfitting). Seeds the gradient-boosting residual idea in [19].



Step function: one constant per leaf region.

## Growing a tree (CART)

The greedy, one-split-at-a-time algorithm sklearn actually runs.

# The CART algorithm

## Grow(node):

1. If pure or a stop rule fires  $\rightarrow$  make a leaf (majority class / mean).
2. Else try every (feature, split) and score by impurity drop.
3. Keep the best split; send rows to the children.
4. **Recurse** on each child.

**Numeric features** can't split "by value" – they need a **threshold** (age > 28?). How that threshold is chosen is the next two slides.

Play Tennis is all-categorical, so the threshold mechanic only shows up on numeric data (e.g. Titanic age / fare).

CART = *Classification And Regression Trees*: same recursion, only the impurity (Gini/entropy vs variance) and the leaf output (class vs mean) change.

## Choosing a split on a numeric feature

A numeric feature needs a **threshold**. The exact recipe:

1. Sort the rows by the feature.
2. Candidate thresholds = **midpoints** between adjacent values.
3. Score each by impurity drop; keep the best.

Example – feature age, label survived (1) / died (0), sorted:

|     |    |    |    |    |    |
|-----|----|----|----|----|----|
| age | 20 | 25 | 30 | 40 | 50 |
| y   | 0  | 0  | 1  | 1  | 1  |

candidate midpoints: 22.5, **27.5**, 35, 45.

Gini gain (parent Gini = 0.48):

| threshold   | gain        |
|-------------|-------------|
| 22.5        | 0.18        |
| <b>27.5</b> | <b>0.48</b> |
| 35          | 0.21        |
| 45          | 0.08        |

age > 27.5 splits it perfectly (two pure leaves). *Shortcut*: only thresholds where the *label changes* can ever win, so you skip the rest.

## Scaling up: quantile and histogram splits

Exact search sorts every feature and tests  $\sim n$  thresholds *at every node* – fine for thousands of rows, painful for millions. Two faster approximations:

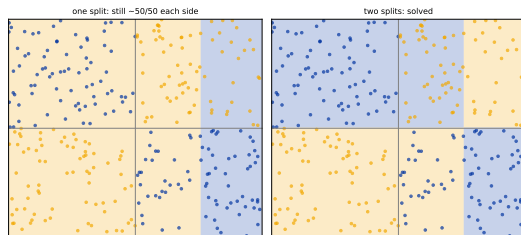
- ▶ **Quantile / percentile splits.** Instead of all midpoints, only test thresholds at the feature's quantiles (e.g. the 32 or 256 percentile cut-points). Far fewer candidates, near-identical splits. (XGBoost's "approx" / weighted quantile sketch.)
- ▶ **Histogram-based splits.** *Bin* each feature into a fixed number of bins (often 255) **once**; a split then only scans bin edges and sums per-bin counts. Cost drops from  $O(n)$  to  $O(\#bins)$ , and the bins are reused at every node. This is how **LightGBM**, **HistGradientBoosting**, and XGBoost (hist) get their speed (callback: the LightGBM frame).

Same idea as exact search – just *fewer* candidate thresholds. A tiny accuracy cost (coarser cut-points) buys a big speed-up on large data; for small data, exact is fine.

## Greedy, not optimal

CART is **greedy**: at each node it takes the split that looks best *right now* and never reconsiders.

- ▶ Finding the *smallest* tree that fits is NP-hard – we cannot search all trees.
- ▶ “Increase purity most” is a cheap **heuristic** for “end up small”.
- ▶ It can **miss a split that only pays off later** – like XOR (right): one split gains nothing, two solve it.



A pure-greedy split sees no gain at step 1; CART still reaches it by growing to depth 2 – but it is a real caveat.

## Overfitting and pruning

A tree left unchecked memorizes the training set; pruning buys back generalization.

## Predict-first: does an unrestricted tree overfit?

Grow with **no depth limit**. What happens to train vs test accuracy?

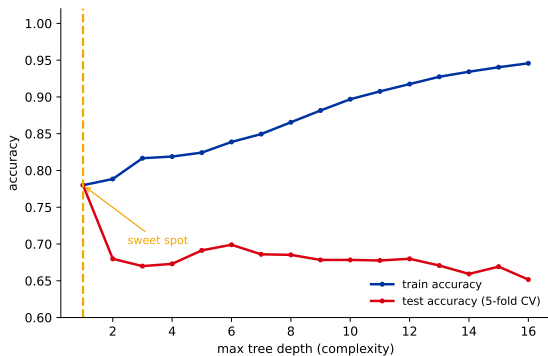
## Predict-first: does an unrestricted tree overfit?

Grow with **no depth limit**. What happens to train vs test accuracy?

On Titanic the unrestricted tree grows to **depth 24, 316 leaves**:

- ▶ train accuracy **96.5%** – near-perfect, it memorizes.
- ▶ test (5-fold CV) only **65%** – *worse* than the depth-1 “sex” stump (78%).

A 30-point train–test gap: textbook overfitting.



Depth is the complexity knob (callback: cross-validation).  
Train keeps rising; test peaks early, then falls.

## Pre-pruning: stop growing early

Set limits *before/while* growing – refuse splits that go too far:

| sklearn parameter     | stops a split when...              |
|-----------------------|------------------------------------|
| max_depth             | the tree is already this deep      |
| min_samples_split     | the node has too few rows to split |
| min_samples_leaf      | a child would be too small         |
| max_leaf_nodes        | the tree already has enough leaves |
| min_impurity_decrease | the split barely helps             |

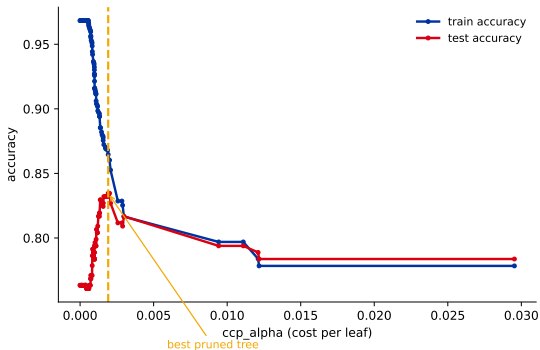
Cheap, needs no extra data. Risk: too aggressive  $\Rightarrow$  underfit; it may stop before a good *later* split.

## Post-pruning: grow full, then cut back

Grow a big tree, then **remove** subtrees that do not earn their keep.

- ▶ **Reduced-error pruning**: cut a subtree if it does not help on a hold-out set.
- ▶ **Cost-complexity pruning** (`ccp_alpha`): minimize  $\text{error} + \alpha (\# \text{leaves})$ . Larger  $\alpha \Rightarrow$  smaller tree.

On Titanic, pruning lifts **test accuracy to 83.5%** at the best `ccp_alpha` – selective cuts beat a uniform depth cap.



Test peaks at the right  $\alpha$ ; over-pruning then underfits.

# Pre- vs post-pruning, and how to choose

## Pre-pruning

- ▶ Fast; no extra data.
- ▶ Can stop too early (misses good later splits).

## Post-pruning

- ▶ Slower; needs data.
- ▶ Can undo bad greedy splits.

**Either way, pick the knob by cross-validation.** Sweep `max_depth` or `ccp_alpha`, keep the value with the best CV score. The depth U-curve is the same picture as the cross-validation lecture's complexity-knob plot.

## Key hyperparameters and their impact

Almost every tree knob controls **one thing – complexity**. Here they are together, with which way each pushes the over/underfit dial. **Tune by CV**.

| hyperparameter                     | what it controls          | raising it $\Rightarrow$                   |
|------------------------------------|---------------------------|--------------------------------------------|
| <code>max_depth</code>             | longest question chain    | more complex ( <b>overfit</b> )            |
| <code>min_samples_leaf</code>      | smallest allowed leaf     | simpler ( <b>regularizes</b> )             |
| <code>min_samples_split</code>     | rows needed to split      | simpler                                    |
| <code>max_leaf_nodes</code>        | total # of leaves         | more complex                               |
| <code>min_impurity_decrease</code> | min gain to allow a split | simpler                                    |
| <code>ccp_alpha</code>             | post-pruning strength     | simpler (more pruned)                      |
| <code>criterion</code>             | gini / entropy            | $\approx$ no accuracy effect (gini faster) |
| <code>max_features</code>          | features tried per split  | $<$ all adds randomness (mainly RF, [18])  |

**In practice:** tune `max_depth` (or `ccp_alpha`) and `min_samples_leaf` by CV – those two do most of the work. `criterion` barely matters; `class_weight="balanced"` handles imbalance (a separate concern, not complexity).

## In practice and bridge

Using trees in sklearn - and why one tree is never the end of the story.

## Decision trees in sklearn (the default tool)

```
from sklearn.tree import DecisionTreeClassifier, plot_tree

tree = DecisionTreeClassifier(max_depth=3, ccp_alpha=0.0,
                              random_state=509)
tree.fit(X_train, y_train)           # Titanic features
plot_tree(tree, feature_names=cols, filled=True)
```

On Titanic the tree usually asks sex first, then an age threshold – readable, you can show it to a stakeholder.

**No scaling needed** – trees only compare thresholds, so monotone rescaling changes nothing (callback: preprocessing). *But* sklearn trees still need **numeric input**: encode categoricals yourself (they are *not* auto-handled, despite folklore).

## CART speaks binary: what one-hot does to the tree

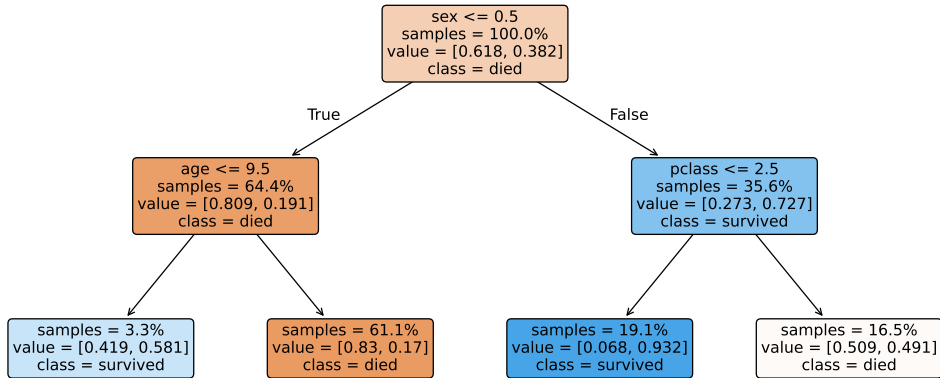
The hand-built tree split `Outlook` **three ways** at once (Sunny / Overcast / Rain) - that is ID3. `sklearn`'s CART only ever asks **one yes/no question** per node.

So you one-hot encode first, and CART recovers the *same decisions* as a chain of binary splits:

`Outlook_Overcast ≤ 0.5 ?` → `Outlook_Sunny ≤ 0.5 ?` → ...

**Same tree, different shape.** Do not be surprised when the tree you *run* looks deeper and asks `Outlook_Overcast ≤ 0.5` instead of a tidy 3-way `Outlook` node - the splits and predictions match, only the drawing differs.

## Titanic: a real (depth-2) tree

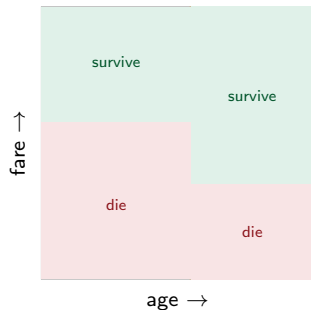


It asks sex first (the strongest split), then age / pclass. Each box shows the split, the % of samples, the class mix, and the colour (orange = died, blue = survived). *Readable enough to hand to a stakeholder – the tree's superpower.*

## The decision boundary is axis-aligned boxes

Every split is “feature  $>$  threshold”, so a tree carves the feature space into **axis-aligned rectangles** – one box per leaf, one constant prediction inside.

The tree on the right and the boxes are the **same model**: walking the tree = finding which box a point falls in.

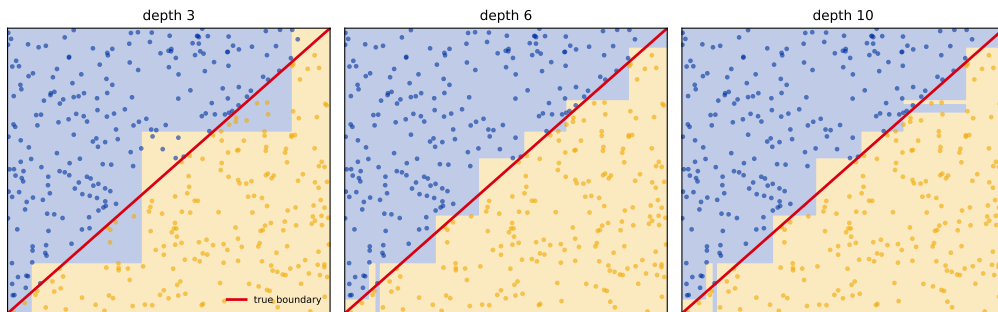


Each leaf = one rectangle.

## Axis-aligned means staircases

A tree can only cut **straight, axis-aligned** lines. A slanted boundary is approximated by a **staircase** – finer (and more overfit) as depth grows.

A diagonal boundary, approximated by axis-aligned steps

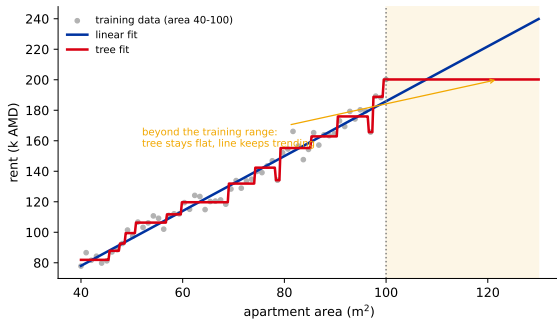


The diagonal (red) is trivial for a line but awkward for a tree – deeper trees chase it with more, smaller boxes. A hint of why we sometimes reach for other models.

# Trees cannot extrapolate

A leaf predicts the **mean of its training rows** – a constant. Outside the training range there is no new leaf, so the prediction **flatlines**.

A linear model keeps trending; a tree *and every tree ensemble* does not. It matters whenever you must predict *beyond* what you have seen (prices, demand, time).



# Strengths and weaknesses

## Strengths

- ▶ Interpretable – read the rules.
- ▶ No scaling; mixed feature types.
- ▶ Captures interactions & non-linearities.
- ▶ Automatic feature selection; fast.
- ▶ Missing values handled natively (no imputation; sklearn  $\geq 1.3$ ).

## Weaknesses

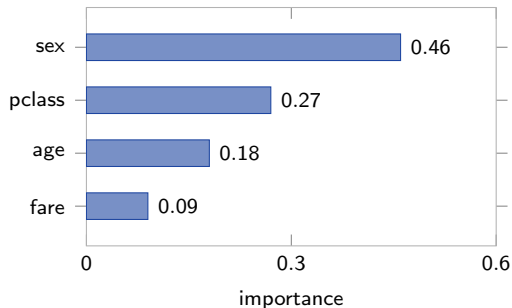
- ▶ **High variance / unstable**: change a few rows  $\rightarrow$  a very different tree.
- ▶ Greedy, not optimal.
- ▶ **Axis-aligned only**: a diagonal boundary needs a staircase of splits.
- ▶ **Never smooth**; extrapolates poorly (flat beyond the data).

**Instability example:** on data like the Wisconsin breast-cancer set, dropping a *single* row can flip many predictions and produce a visibly different tree. This variance is the headline weakness – and the reason ensembles exist.

## Feature importance (read with care)

A tree scores each feature by **how much impurity its splits removed** (summed over the tree). Quick way to see what drove the model.

**Caveat:** this impurity-based importance is **biased toward high-cardinality** features (many split points). Prefer *permutation importance*; SHAP comes later (interpretability).



Titanic: sex dominates.

# The tree is the atom of this chapter

A single tree is weak (high variance). Everything next is built from it.

*# a single tree, two ways:*

`DecisionTreeRegressor(max_depth=3)`

*# sklearn*

`LGBMRegressor(n_estimators=1, learning_rate=1.0)`

*# LightGBM*

**Why** `learning_rate=1.0`? LightGBM is a *boosting library*; with the default 0.1 it would shrink the single tree. It also grows *leaf-wise* (knob `num_leaves`) with histogram bins, so it is not byte-identical to sklearn's CART – same idea, different growth.

1 tree → **bag many** = Random Forest ([18]) → **boost** them = Gradient Boosting / XGBoost / LightGBM ([19]–[20]).

# Recap

- ▶ A decision tree = **nested yes/no questions**; a leaf gives the prediction (majority class or mean).
- ▶ Grow greedily (**CART**): at each node pick the split that drops **impurity** most (**Gini**/entropy for classification, variance for regression).
- ▶ Unlimited depth  $\Rightarrow$  0 training error but **overfit**; control with **pre-pruning** and **post-pruning** (`ccp_alpha`), chosen by **CV**.
- ▶ Strengths: interpretable, no scaling. Weakness: **high variance**.

**Workflow:** encode categoricals  $\rightarrow$  fit a shallow tree as an interpretable baseline  $\rightarrow$  tune `max_depth/ccp_alpha` by CV  $\rightarrow$  read the rules / importances  $\rightarrow$  if you need accuracy over interpretability, reach for ensembles ([18]–[20]).